

1-Number of Zeros in a Given Array

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s.

Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer:(penalty regime: 0 %)

```
1  #include <stdio.h>
2
3  int countZeroes(int arr[], int low, int high, int n)
4  {
5      if (low > high)
6          return 0;
7
8      int mid = low + (high - low) / 2;
9
10     // First 0 is found
11     if (arr[mid] == 0 && (mid == 0 || arr[mid - 1] == 1))
12         return n - mid;
13
14     // If middle element is 1, search right half
15     if (arr[mid] == 1)
16         return countZeroes(arr, mid + 1, high, n);
17
18     // If middle element is 0, search left half
19     return countZeroes(arr, low, mid - 1, n);
20 }
21
```

```

22 int main()
23 {
24     int m;
25     scanf("%d", &m);
26
27     int arr[m];
28
29     for (int i = 0; i < m; i++)
30         scanf("%d", &arr[i]);
31
32     printf("%d\n", countZeroes(arr, 0, m - 1, m));
33
34     return 0;
35 }

```

	Input	Expected	Got	
✓	5	2	2	✓
	1			
	1			
	1			
	0			
	0			

[illegible]

✓	17	2	2	✓
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	0			
	0			

Passed all tests! ✓

2-Majority Element

Given an array `nums` of size `n`, return *the majority element*.
 The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:
Input: `nums = [3,2,3]`

Output: 3

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`

Output: 2

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 104`
- `-231 <= nums[i] <= 231 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer:(penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int countOccurrences(int arr[], int left, int right, int element)
4 {
5     int count = 0;
6
7     for (int i = left; i <= right; i++)
8     {
9         if (arr[i] == element)
10             count++;
11     }
12
13     return count;
14 }
15
16 int majorityElement(int arr[], int left, int right)
17 {
18
19     if (left == right)
20         return arr[left];
21
22     int mid = left + (right - left) / 2;
23
24     int leftMajority = majorityElement(arr, left, mid);
25     int rightMajority = majorityElement(arr, mid + 1, right);
26
27
28     if (leftMajority == rightMajority)
29         return leftMajority;
30
31
32     int leftCount = countOccurrences(arr, left, right, leftMajority);
33     int rightCount = countOccurrences(arr, left, right, rightMajority);
34
35     return (leftCount > rightCount) ? leftMajority : rightMajority;
36 }
37
```

```

38 int main()
39 {
40     int n;
41     scanf("%d", &n);
42
43     int nums[n];
44
45     for (int i = 0; i < n; i++)
46         scanf("%d", &nums[i]);
47
48     printf("%d\n", majorityElement(nums, 0, n - 1));
49
50     return 0;
51 }

```

	Input	Expected	Got	
✓	3	3	3	✓
	3 2 3			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

3-Finding Floor Value

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array
Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer:(penalty regime: 0 %)

```
1  #include <stdio.h>
2
3  int findFloor(int arr[], int low, int high, int x)
4  {
5      if (low > high)
6          return -1;
7
8      if (arr[high] <= x)
9          return arr[high];
10
11     if (arr[low] > x)
12         return -1;
13
14     int mid = low + (high - low) / 2;
15
16     if (arr[mid] == x)
17         return arr[mid];
18
19     if (arr[mid] < x)
20     {
21         int result = findFloor(arr, mid + 1, high, x);
22
23         if (result != -1)
24             return result;
25
26         return arr[mid];
27     }
28
29     return findFloor(arr, low, mid - 1, x);
30 }
```

```

31
32 int main()
33 {
34     int n, x;
35
36     scanf("%d", &n);
37
38     int arr[n];
39
40     for (int i = 0; i < n; i++)
41         scanf("%d", &arr[i]);
42
43     scanf("%d", &x);
44
45     printf("%d\n", findFloor(arr, 0, n - 1, x));
46
47     return 0;
48 }

```

	Input	Expected	Got	
✓	6	2	2	✓
	1			
	2			
	8			
	10			
	12			
	19			
	5			

✓	5	85	85	✓
	10			
	22			
	85			
	108			
	129			
	100			
✓	7	9	9	✓
	3			
	5			
	7			
	9			
	11			
	13			
	15			
	10			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

4-Two Elements sum to x

Problem Statement:

Given a sorted array of integers say `arr[]` and a number `x`. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = `x`. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value “x”)

Answer:(penalty regime: 0 %)

```
1  #include <stdio.h>
2
3  int findPair(int arr[], int left, int right, int x, int *a, int *b)
4  {
5      if (left >= right)
6          return 0;
7
8      int sum = arr[left] + arr[right];
9
10     if (sum == x)
11     {
12         *a = arr[left];
13         *b = arr[right];
14         return 1;
15     }
16
17     if (sum < x)
18         return findPair(arr, left + 1, right, x, a, b);
19
20     return findPair(arr, left, right - 1, x, a, b);
21 }
22
23 int main()
24 {
25     int n, x;
26     scanf("%d", &n);
27
28     int arr[n];
29
30     for (int i = 0; i < n; i++)
31         scanf("%d", &arr[i]);
32 }
```

```

33     scanf("%d", &x);
34
35     int a, b;
36
37     if (findPair(arr, 0, n - 1, x, &a, &b))
38     {
39         printf("%d\n", a);
40         printf("%d\n", b);
41     }
42     else
43     {
44         printf("No\n");
45     }
46
47     return 0;
48 }

```

	Input	Expected	Got	
✓	4	4	4	✓
	2	10	10	
	4			
	8			
	10			
	14			

✓	5	No	No	✓
	2			
	4			
	6			
	8			
	10			
	100			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

5-Implementation of Quick Sort

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

```
1  #include <stdio.h>
2
3  void swap(int *a, int *b)
4  {
5      int temp = *a;
6      *a = *b;
7      *b = temp;
8  }
9
10 int partition(int arr[], int low, int high)
11 {
12     int pivot = arr[high];
13     int i = low - 1;
14
15     for (int j = low; j < high; j++)
16     {
17         if (arr[j] < pivot)
18         {
19             i++;
20             swap(&arr[i], &arr[j]);
21         }
22     }
23
24     swap(&arr[i + 1], &arr[high]);
25
26     return i + 1;
27 }
28
29 void quickSort(int arr[], int low, int high)
30 {
31     if (low < high)
32     {
33         int pi = partition(arr, low, high);
34
35         quickSort(arr, low, pi - 1);
36         quickSort(arr, pi + 1, high);
37     }
38 }
39
```

```

39
40 int main()
41 {
42     int n;
43     scanf("%d", &n);
44
45     int arr[n];
46
47     for (int i = 0; i < n; i++)
48         scanf("%d", &arr[i]);
49
50     quickSort(arr, 0, n - 1);
51
52     for (int i = 0; i < n; i++)
53     {
54         if (i > 0)
55             printf(" ");
56         printf("%d", arr[i]);
57     }
58
59     return 0;
60 }

```

	Input	Expected	Got	
✓	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✓
✓	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✓
✓	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.